

Documentación de la Base de Datos

Subgrupo: Base de Datos e Interfaz

Versión de esquema: 2

Motor: SQLite

1. Alcance y propósito de este documento

Este documento describe el esquema de datos que ejecuta el ESP32 para la máquina expendedora del proyecto: qué tablas existen, para qué sirve cada una, sus atributos con tipo de dato y restricciones, y cómo se relacionan entre sí. Corresponde al primer entregable del subgrupo de Base de Datos (crear, consultar, reiniciar y conservar registros) y sirve como referencia para Programación ESP32, Arquitectura y QA.

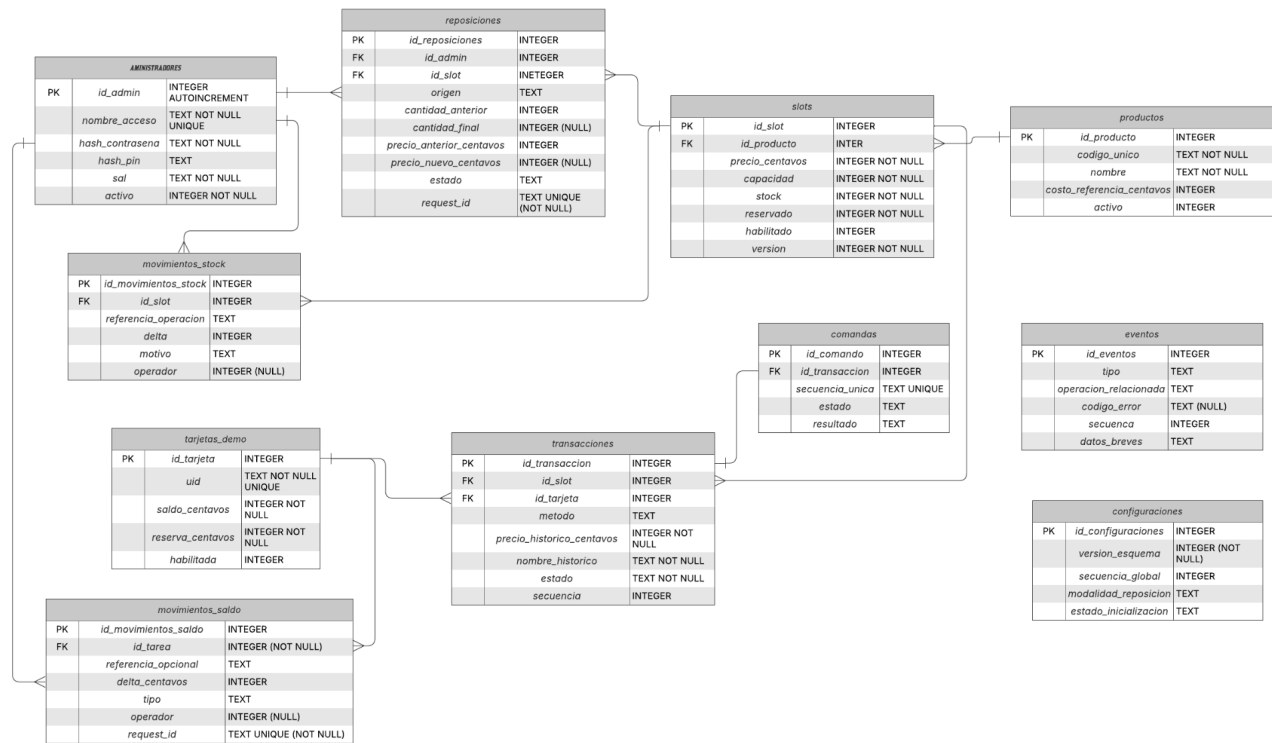
El script SQL correspondiente a este esquema se entrega junto con este documento como archivo separado (schema.sql) y fue validado ejecutándose contra un motor SQLite real antes de esta entrega.

2. Convenciones generales del esquema

- El dinero se guarda siempre como número entero en centavos (columnas *_centavos), nunca como decimal, para evitar errores de redondeo.
- Las cantidades (stock, capacidad, reservado) son enteras.
- SQLite no tiene tipo booleano nativo: los valores de sí/no se representan como INTEGER restringido a 0 o 1 mediante CHECK (activo, habilitado, habilitada).
- Ningún registro con historial se borra físicamente: se desactiva con una bandera (activo = 0, habilitado = 0, habilitada = 0).
- Toda operación que debe poder repetirse sin duplicar su efecto (reposición, recarga) tiene una columna request_id con restricción UNIQUE. Esto es lo que hace la operación idempotente.
- No existen fechas de calendario reales: el orden de los eventos se resuelve con contadores de secuencia y tiempo transcurrido desde el arranque, ya que el sistema puede operar sin Internet ni reloj confiable.
- Las claves foráneas están activas (PRAGMA foreign_keys = ON) y se comprueban en cada operación.

3. Diagrama entidad-relación

El diagrama siguiente muestra las 14 tablas del esquema y sus relaciones. El color indica el tipo de tabla: azul claro para catálogos/entidades base, naranja para el ciclo de venta, verde para bitácoras de auditoría y gris para configuración/diagnóstico.



4. Documentación detallada por tabla

4.1 administradores

Cuentas de las personas autorizadas a realizar mantenimiento, reposición y recargas. Es la tabla que da identidad y trazabilidad a cualquier operación administrativa del sistema.

Atributo	Tipo SQLite	Llave / Restricción	Descripción
id_admin	INTEGER	PK, AUTOINCREMENT	Identificador interno del administrador.
nombre_acceso	TEXT	NOT NULL, UNIQUE	Usuario con el que inicia sesión (por ejemplo, desde la interfaz web).
hash_contrasena	TEXT	NOT NULL	Resultado de aplicar una función hash a la contraseña combinada con la sal. Nunca se guarda la contraseña en texto plano.
hash_pin	TEXT	—	Resultado de aplicar hash al PIN usado para entrar desde el teclado físico de la máquina.
sal	TEXT	NOT NULL	Valor aleatorio único por administrador, usado junto con la contraseña/PIN antes de calcular el hash, para que dos personas con la misma contraseña no generen el mismo hash.
activo	INTEGER	NOT NULL, CHECK IN (0,1)	Baja lógica: 0 deshabilita al administrador sin borrar su historial.

Notas de diseño:

- Un PIN corto compartido entre varios administradores aumenta el riesgo de que alguien lo adivine por accidente; por eso el PIN es individual y no un código único general.
- Esta tabla es referenciada por reposiciones (autor de la reposición) y por movimientos_stock / movimientos_saldo (columna operador) para poder saber siempre quién hizo cada operación.

4.2 productos

Catálogo de productos que pueden ofrecerse en la máquina, independiente de en qué canal físico están colocados en un momento dado.

Atributo	Tipo SQLite	Llave / Restricción	Descripción
id_producto	INTEGER	PK, AUTOINCREMENT	Identificador interno del producto.
codigo_unico	TEXT	NOT NULL, UNIQUE	Código de catálogo del producto.
nombre	TEXT	NOT NULL	Nombre visible del producto.
costo_referencia_centavos	INTEGER	–	Costo de referencia opcional, informativo; no es el precio de venta (ese vive en slots).
activo	INTEGER	NOT NULL, CHECK IN (0,1)	Baja lógica del producto.

Notas de diseño:

- El precio de venta no se guarda aquí porque puede variar según el canal; por eso vive en slots.precio_centavos.
- Existe una operación específica para reasignar el producto de un canal (POST /api/slots/{id}/product) en lugar de dar de baja y crear un registro nuevo, porque el canal es la posición física estable y debe conservar su historial de ventas.

4.3 slots

Los 4 canales físicos de la máquina. El canal es una posición fija; el producto que contiene puede cambiarse en modo mantenimiento.

Atributo	Tipo SQLite	Llave / Restricción	Descripción
id_slot	INTEGER	PK, CHECK BETWEEN 1 AND 4	Número de canal físico (1 a 4).
id_producto	INTEGER	FK → productos.id_producto	Producto actualmente asignado al canal; puede ser NULL si está vacío.
precio_centavos	INTEGER	NOT NULL, CHECK >= 0	Precio de venta vigente en este canal.
capacidad	INTEGER	NOT NULL, CHECK >= 0	Máximo de unidades que caben físicamente en el canal.
stock	INTEGER	NOT NULL, CHECK >= 0	Unidades disponibles actualmente.
reservado	INTEGER	NOT NULL, CHECK >= 0	Unidades apartadas por una compra en curso, aún no confirmada.
habilitado	INTEGER	NOT NULL, CHECK IN (0,1)	Permite deshabilitar el canal sin perder su historial.
version	INTEGER	NOT NULL	Contador que se incrementa en cada cambio del canal; se usa para detectar si otra operación modificó el canal mientras se leía (conurrencia optimista).

Notas de diseño:

- Restricción CHECK (stock + reservado <= capacidad): impide guardar un estado físicamente imposible.

- El stock vive únicamente aquí; movimientos_stock es historial, no otra fuente de verdad.

4.4 tarjetas_demo

Tarjetas RFID simuladas usadas como método de pago dentro del proyecto.

Atributo	Tipo SQLite	Llave / Restricción	Descripción
id_tarjeta	INTEGER	PK, AUTOINCREMENT	Identificador interno de la tarjeta.
uid	TEXT	NOT NULL, UNIQUE	Identificador físico leído por el lector RFID.
saldo_centavos	INTEGER	NOT NULL, CHECK >= 0	Saldo disponible de la tarjeta.
reserva_centavos	INTEGER	NOT NULL, CHECK >= 0	Monto apartado por una compra en curso.
habilitada	INTEGER	NOT NULL, CHECK IN (0,1)	Baja lógica de la tarjeta.

Notas de diseño:

- Restricción CHECK (saldo_centavos - reserva_centavos >= 0): el saldo disponible real nunca puede quedar negativo.

4.5 transacciones

Registro central del ciclo de vida de cada compra, desde que se autoriza hasta que se confirma o queda incierta.

Atributo	Tipo SQLite	Llave / Restricción	Descripción
id_transaccion	INTEGER	PK, AUTOINCREMENT	Identificador de la compra.
id_slot	INTEGER	NOT NULL, FK → slots.id_slot	Canal donde ocurrió la compra.
id_tarjeta	INTEGER	FK → tarjetas_demo.id_tarjeta (NULL)	Tarjeta usada para pagar; NULL si el pago fue simulado sin tarjeta.
metodo	TEXT	NOT NULL	Método de pago usado (por ejemplo, efectivo o tarjeta).
precio_historico_centavos	INTEGER	NOT NULL, CHECK >= 0	Copia del precio del producto en el momento exacto de la venta.
nombre_historico	TEXT	NOT NULL	Copia del nombre del producto en el momento exacto de la venta.
estado	TEXT	NOT NULL, CHECK IN ('RESERVADA', 'CONFIRMADA', 'INCIERTA')	Etapas actuales del ciclo de vida de la compra.
secuencia	INTEGER	—	Orden lógico del evento, usado en lugar de una fecha real.

Notas de diseño:

- precio_historico_centavos y nombre_historico son una duplicación intencional (no un error de normalización): preservan lo que realmente se vendió aunque después cambie el producto o el precio del canal.

- id_tarjeta es opcional a propósito, porque el sistema permite compras simuladas sin tarjeta.

4.6 comandos

La orden física de venta ("VEND") enviada al motor del canal correspondiente, y el resultado que reporta el hardware.

Atributo	Tipo SQLite	Llave / Restricción	Descripción
id_comando	INTEGER	PK, AUTOINCREMENT	Identificador del comando.
id_transaccion	INTEGER	NOT NULL, FK → transacciones. id_transaccion , UNIQUE	Compra a la que pertenece este comando.
secuencia_unica	TEXT	UNIQUE	Identificador único de este comando dentro del flujo de ejecución.
estado	TEXT	—	Estado de envío del comando (pendiente, enviado, respondido).
resultado	TEXT	CHECK IN ('DELIVERED', 'REJECTED_BEFORE_MOTION', 'UNCERTAIN') o NULL	Veredicto final reportado por el hardware.

Notas de diseño:

- El UNIQUE sobre id_transaccion obliga la regla de negocio "un VEND por transacción": es imposible mandar dos veces el motor por la misma compra.
- Esta tabla separa la decisión de negocio (transacciones) de lo que realmente hizo el mecanismo físico, lo cual es clave para diagnosticar el estado INCIERTA.

4.7 reposiciones

Cada operación de reabastecimiento de un canal, desde que se solicita hasta que se cierra con la cantidad final contada.

Atributo	Tipo SQLite	Llave / Restricción	Descripción
id_reposicion	INTEGER	PK, AUTOINCREMENT	Identificador de la reposición.
id_admin	INTEGER	NOT NULL, FK → administrado res.id_admin	Quién realizó la reposición.
id_slot	INTEGER	NOT NULL, FK → slots.id_slot	Canal reabastecido.
origen	TEXT	NOT NULL, CHECK IN ('teclado', 'web')	Vía por la que se realizó la reposición.
cantidad_anterior	INTEGER	NOT NULL	Unidades registradas antes de reponer.

Atributo	Tipo SQLite	Llave / Restricción	Descripción
cantidad_final	INTEGER	NULL hasta el cierre	Unidades contadas físicamente al cerrar la reposición.
precio_anterior_centavos	INTEGER	NOT NULL	Precio vigente antes de la reposición.
precio_nuevo_centavos	INTEGER	NULL = se conserva el mismo	Nuevo precio, si el administrador decidió cambiarlo.
estado	TEXT	NOT NULL, CHECK IN ('PENDIENTE', 'CERRADA')	Etapas de la reposición.
request_id	TEXT	NOT NULL, UNIQUE	Identificador de idempotencia de la operación.

Notas de diseño:

- El registro se crea en `iniciarReposicion` antes de la manipulación física del canal, para que sobreviva un reinicio del ESP32 a la mitad del proceso.
- `request_id` UNIQUE garantiza que, si se repite la confirmación, se devuelva el resultado ya guardado en vez de aplicar el ajuste una segunda vez.

4.8 movimientos_stock

Bitácora de auditoría de cada cambio de stock ocurrido en un canal (venta, reposición o ajuste manual).

Atributo	Tipo SQLite	Llave / Restricción	Descripción
id_movimiento_stock	INTEGER	PK, AUTOINCREMENT	Identificador del movimiento.
id_slot	INTEGER	NOT NULL, FK → slots.id_slot	Canal afectado.
referencia_operacion	TEXT	—	Referencia libre a la operación que originó el movimiento (por ejemplo, el id de una transacción o reposición); no es llave foránea porque puede apuntar a distintas tablas según el caso.
delta	INTEGER	NOT NULL	Cambio aplicado al stock: negativo en venta, positivo en reposición.
motivo	TEXT	NOT NULL, CHECK IN ('VENTA', 'REPOSICION', 'AJUSTE')	Causa del movimiento.
operador	INTEGER	FK → administradores.id_admin (NULL)	Administrador responsable; NULL si el movimiento fue automático (por ejemplo, una venta).

Notas de diseño:

- Esta tabla es de solo lectura una vez escrita (append-only): no reemplaza a `slots.stock` como fuente de verdad, es evidencia histórica para auditoría.

4.9 movimientos_saldo

Bitácora de auditoría de cada cambio de saldo de una tarjeta (compras pagadas con tarjeta y recargas).

Atributo	Tipo SQLite	Llave / Restricción	Descripción
id_movimiento_saldo	INTEGER	PK, AUTOINCREMENT	Identificador del movimiento.
id_tarjeta	INTEGER	NOT NULL, FK → tarjetas_demo.id_tarjeta	Tarjeta afectada. (Corregido: el diagrama original tenía un error de captura como id_tarea.)
referencia_opcional	TEXT	—	Referencia libre a la operación relacionada (por ejemplo, una transacción), sin llave foránea formal.
delta_centavos	INTEGER	NOT NULL	Cambio aplicado al saldo: negativo en compra, positivo en recarga.
tipo	TEXT	NOT NULL, CHECK IN ('COMPRA', 'RECARGA')	Naturaleza del movimiento.
operador	INTEGER	FK → administradores.id_admin (NULL)	Administrador que autorizó la recarga; NULL si el movimiento fue una compra automática.
request_id	TEXT	NOT NULL, UNIQUE	Identificador de idempotencia de la operación.

Notas de diseño:

- request_id UNIQUE es lo que garantiza que recargarTarjeta produzca un único movimiento de saldo aunque la petición se repita.

4.10 eventos

Bitácora general de diagnóstico (errores y advertencias del sistema), independiente de si una operación de negocio se completó o no.

Atributo	Tipo SQLite	Llave / Restricción	Descripción
id_evento	INTEGER	PK, AUTOINCREMENT	Identificador del evento.
tipo	TEXT	NOT NULL, CHECK IN ('ERROR', 'INFO', 'ADVERTENCIA')	Clasificación del evento.
operacion_relacionada	TEXT	—	Referencia libre a la operación asociada, sin llave foránea formal.
codigo_error	TEXT	NULL	Código de error, si aplica.
secuencia	INTEGER	—	Orden lógico del evento.
datos_breves	TEXT	—	Resumen corto del evento; no un volcado completo de datos.

Notas de diseño:

- Se usa para diagnosticar fallas graves (base ilegible, sin espacio) sin depender de que la operación relacionada haya llegado a completarse.

4.11 configuraciones

Estado global del sistema, consultado por el ESP32 al arrancar.

Atributo	Tipo SQLite	Llave / Restricción	Descripción
id_configuracion	INTEGER	PK, AUTOINCREMENT	Se espera una única fila activa (id = 1).
version_esquema	INTEGER	NOT NULL	Versión actual del esquema, para detectar si hace falta migrar.
secuencia_global	INTEGER	—	Contador global usado para ordenar eventos sin reloj de calendario.
modalidad_reposicion	TEXT	CHECK IN ('teclado', 'web') o NULL	Vía de reposición activa, definida en el hito D7 del proyecto.
estado_inicializacion	TEXT	—	Indica si ya se cargaron los datos semilla iniciales.

4.12 denominaciones

Catálogo de los valores de efectivo (monedas y billetes) que el teclado permite seleccionar para simular un pago en efectivo. No representa un validador físico de billetes o monedas: la máquina no acepta efectivo real, según lo definido por Arquitectura.

Atributo	Tipo SQLite	Llave / Restricción	Descripción
id_denominacion	INTEGER	PK, AUTOINCREMENT	Identificador interno de la denominación.
valor_centavos	INTEGER	NOT NULL, CHECK > 0	Valor nominal de la denominación, en centavos.
tipo	TEXT	NOT NULL, CHECK IN ('moneda', 'billete')	Distingue monedas de billetes; necesario porque el valor \$20 existe en ambas formas en pesos mexicanos.
activa	INTEGER	NOT NULL, CHECK IN (0,1)	Baja lógica de la denominación.

Notas de diseño:

- La combinación (valor_centavos, tipo) es UNIQUE, no solo valor_centavos, porque el peso mexicano tiene denominaciones de \$20 tanto en moneda como en billete.
- Esta tabla es referenciada por caja_efectivo (cantidad disponible) y por movimientos_efectivo (bitácora de cada cambio de esa cantidad).

4.13 caja_efectivo

Cantidad actual disponible de cada denominación dentro de la máquina, usada para verificar si hay suficiente para dar cambio antes de confirmar una venta en efectivo. Es la fuente de verdad de la cantidad; no se duplica en otra tabla editable.

Atributo	Tipo SQLite	Llave / Restricción	Descripción
id_denominacion	INTEGER	PK, FK → denominaciones.id_denominacion	Denominación a la que pertenece este registro de cantidad.
cantidad	INTEGER	NOT NULL, CHECK >= 0	Piezas disponibles actualmente en la máquina, tanto para completar pagos como para dar cambio.
version	INTEGER	NOT NULL	Contador de concurrencia optimista, igual que slots.version: detecta si un ajuste desde la web y el cierre de una venta modifican la cantidad al mismo tiempo.

Notas de diseño:

- La modificación de cantidad desde la web (por ejemplo, para reponer monedas físicamente y así poder dar cambio) actualiza esta tabla directamente y deja evidencia en movimientos_efectivo con motivo AJUSTE.

4.14 movimientos_efectivo

Bitácora de auditoría de cada cambio en caja_efectivo: piezas recibidas como parte de un pago, piezas entregadas como cambio, o ajustes manuales del administrador desde la web. Una sola tabla cubre tanto el detalle de composición de un pago en efectivo (varias piezas por transacción) como el registro de auditoría de la cantidad disponible, para no duplicar la misma información en dos tablas distintas.

Atributo	Tipo SQLite	Llave / Restricción	Descripción
id_movimiento_efectivo	INTEGER	PK, AUTOINCREMENT	Identificador del movimiento.
id_denominacion	INTEGER	NOT NULL, FK → denominaciones.id_denominacion	Denominación afectada por el movimiento.
id_transaccion	INTEGER	FK → transacciones.id_transaccion (NULL)	Compra que originó el movimiento; NULL si es un ajuste manual sin venta asociada.
delta	INTEGER	NOT NULL	Piezas agregadas (positivo, pago recibido) o retiradas (negativo, cambio entregado o ajuste) de caja_efectivo.
motivo	TEXT	NOT NULL, CHECK IN ('PAGO_RECIBIDO', 'CAMBIO_ENTREGADO', 'AJUSTE')	Causa del movimiento.
operador	INTEGER	FK → administradores.id_admin (NULL)	Administrador responsable si fue un ajuste manual; NULL si fue automático (parte de una venta).
request_id	TEXT	UNIQUE (NULL)	Identificador de idempotencia; en la práctica solo se manda en ajustes hechos desde la web.
secuencia	INTEGER	—	Orden lógico del movimiento dentro de una misma transacción (por ejemplo, primero el pago, después el cambio).

Notas de diseño:

- Esta tabla reemplaza lo que en un borrador previo eran dos tablas separadas (efectivo_insertado y movimientos_efectivo): mantenerlas separadas hubiera duplicado el mismo hecho (qué pieza se usó y en qué transacción) en dos lugares distintos.
- Al cerrar una venta en efectivo, se inserta una fila por cada denominación recibida (PAGO_RECIBIDO) y una por cada denominación devuelta como cambio (CAMBIO_ENTREGADO), y se actualiza caja_efectivo en la misma transacción SQL, igual que guardarReposicion aplica stock, precio y movimiento juntos.

5. Relaciones entre tablas

Todas las demás relaciones del esquema son de uno a muchos (1:N); no existe ninguna relación de muchos a muchos. Dos relaciones se comportan como uno a uno (1:1) en la práctica: transacciones-comandos, forzada mediante una restricción UNIQUE, y denominaciones-caja_efectivo, forzada porque id_denominacion es a la vez PK y FK.

Relación (padre → hijo)	Cardinalidad	¿Obligatoria?	Motivo
productos → slots	1 : N	Opcional	Un slot puede no tener producto asignado (canal vacío).
administradores → reposiciones	1 : N	Obligatoria	Toda reposición fue realizada por un administrador.
administradores → movimientos_stock	1 : N	Opcional	El movimiento puede ser automático (venta), sin operador humano.
administradores → movimientos_saldo	1 : N	Opcional	Igual que el anterior, para movimientos de saldo.
slots → transacciones	1 : N	Obligatoria	Toda compra ocurre en un canal específico.
slots → reposiciones	1 : N	Obligatoria	Toda reposición pertenece a un canal específico.
slots → movimientos_stock	1 : N	Obligatoria	Todo movimiento de stock pertenece a un canal.
tarjetas_demo → transacciones	1 : N	Opcional	El pago puede ser simulado sin tarjeta (efectivo).
tarjetas_demo → movimientos_saldo	1 : N	Obligatoria	Todo movimiento de saldo pertenece a una tarjeta.
transacciones → comandos	1 : 1 (forzada con UNIQUE)	Opcional del lado comandos	Regla de negocio: un único VEND por transacción.
denominaciones → caja_efectivo	1 : 1	Obligatoria	Cada denominación activa tiene exactamente un registro de cantidad disponible.
denominaciones → movimientos_efectivo	1 : N	Obligatoria	Todo movimiento de efectivo pertenece a una denominación específica.
transacciones → movimientos_efectivo	1 : N	Opcional	Solo las ventas pagadas en efectivo generan movimientos; los ajustes manuales no tienen transacción asociada.
administradores → movimientos_efectivo	1 : N	Opcional	El movimiento puede ser automático (parte de una venta), sin operador humano.

Referencias libres (no son llaves foráneas formales, por diseño): movimientos_stock.referencia_operacion, movimientos_saldo.referencia_opcional y eventos.operacion_relacionada. Son campos de texto que apuntan a distintas tablas según el caso, pensados para lectura/diagnóstico, no para integridad referencial.

6. Historial de correcciones sobre el diagrama original

- movimientos_saldo: la columna se llamaba id_tarea por error de captura; se corrigió a id_tarjeta como llave foránea hacia tarjetas_demo.
- movimientos_stock.operador y movimientos_saldo.operador: se formalizaron como llave foránea hacia administradores.id_admin (antes eran una columna suelta sin restricción declarada).
- La tabla "comandas" del diagrama se documentó como comandos, por consistencia con el nombre usado en el encargo oficial del proyecto.
- Se agregaron las tablas denominaciones, caja_efectivo y movimientos_efectivo para soportar el pago simulado en efectivo por teclado (sin aceptador físico de monedas) y su cambio: composición del pago, cantidad disponible para dar cambio, y ajustes manuales desde la web.

7. Control de versiones de este documento

Versión	Fecha / hito	Cambios
1	Hito D2–D3	Primera versión del esquema, validada contra un motor SQLite real (schema.sql adjunto).
2	Hito D3–D5	Se agregó soporte de pago simulado en efectivo: tablas denominaciones, caja_efectivo y movimientos_efectivo (14 tablas en total). schema.sql y seed.sql actualizados.